Spring 2026
DD2373 Automata and Languages
Lab Assignment 1:
Regular Expressions and Text Search

Andreas Lundblad Dilian Gurov

March 23, 2026

1 Introduction

The objective of this lab is to develop a tool similar to the UNIX `grep` command. Given a regular expression and a text, your program should filter out and print those lines of the text that contain at least one substring that matches the regular expression. The task is accomplished by carrying out the following steps:

1. Read and parse the regular expression;
2. Construct the corresponding ϵ-NFA;
3. Convert the ϵ-NFA into an equivalent DFA;
4. Minimize the DFA;
5. Simulate the resulting DFA, with lines from the text file as input.

You are free to choose the programming language in which you implement the algorithms. Do not use any built-in or third-party regular expression libraries for this task. However, for those who chose Java or Haskell, parsers of regular expressions will be provided. (see section 4)

The assignment is carried out in teams of at most two persons.

AI Notice

AI-tools may be used to any extent to create parsers equivalent to those provided in Java and Haskell (see section 4). E.g., you are free to prompt an LLM-based tool to convert the provided parsers into another language. For the other parts of the lab, the general AI regulations of the course apply (see Canvas).

2 Regular Expressions

In this lab we shall focus on a subset of the UNIX regular expressions described in section 3.3.1 in the course book. The syntax of the expressions you should be able to handle is described by the following BNF grammar:

$$R ::= a \mid \texttt{.} \mid R_1 \texttt{|} R_2 \mid R_1 R_2 \mid R\texttt{*} \mid R\texttt{+} \mid R\texttt{?} \mid \texttt{(}R\texttt{)}$$

where a ranges over Σ (in this case, a subset of the printable ASCII characters [1]). The expressions are interpreted according to Table 1.

Expression	Matches
a	the corresponding ASCII character
`.`	any character
R_1`|`R_2	R_1 or R_2
$R_1 R_2$	R_1 followed by R_2
R`*`	any number of repetitions of R
R`+`	one or more repetitions of R
R`?`	zero or one repetition of R
`(`R`)`	R

Table 1: Interpretation of this lab's regular expressions.

3 Implementation

3.1 Reading and parsing the regular expression

As previously mentioned, a parser is readily available for Java and Haskell users (notice that the "star" operator R`*` is called "closure" in the implementation). Those of you who wish to implement a parser on your own, are advised to use a parser generator such as `flex`.

[1] For simplicity, you may assume that Σ does not contain special characters: "`.`", "`|`", "`*`", "`+`", "`?`", "`(`" and "`)`".

3.2 Construction of the ϵ-NFA

Theorem 3.7 in the course book states that for each regular expression there exists an ϵ-NFA accepting the same language. The proof presented in class and in the book is constructive: it uses an inductive construction, which produces from a regular expression a corresponding ϵ-NFA. Reading through and understanding this proof will help you carry out this step.

3.3 Converting the ϵ-NFA into an equivalent DFA

The construction converting an ϵ-NFA into an equivalent DFA is described in sections 2.5.3 till 2.5.5 of the course book.

3.4 Minimizing the DFA

The previous step is a subset construction that usually results in large automata with many equivalent states. The obtained DFA should in this step be minimized by applying the table–filling algorithm described in class and in section 4.4.1 of the book (or alternatively, with the algorithm based on iterative computation of $\approx_k$ discussed in class) to compute the equivalent states of this DFA, and by constructing the quotient automaton described in class and in section 4.4.3 of the book.

3.5 Simulating the DFA

To search for strings matching the original regular expression, the DFA should be simulated. Your tool should inspect the input text line by line and look for substrings matching the regular expression.

4 Resources

The Java (or Haskell) source code for the regular expression parser and some test cases are available at: `https://gits-15.sys.kth.se/dd2372/lab1` (or from the canvas assignment page). The test case files are organized as follows: The first line contains the relevant input symbols (the alphabet, or Σ), the second line contains the regular expression in the format described in section 2, and the remaining lines constitute the input that should be processed. (For an example test case file, see section 6.)

5 Tasks

The present lab assignment consists of the following tasks:

1. Implement the tool described above. Include the functionality of printing out the automata (graphically, or in table form) constructed at each individual step; this will be useful for debugging and demonstrating the tool.

2. Construct and run a few small as well as large test cases of your own.

3. Write a succinct report describing your findings. For instance, how did you decide to represent your transition relation? How many states do the intermediate automata have in your test cases? Is performance affected significantly by the sizes of the two inputs? In the last step of searching for the pattern, couldn't one instead just simulate directly the ϵ-NFA, without determinizing and minimizing it first?

6 Example Run

```
$ cat input.txt
abcxy                          The test case input language (Σ).
(aba)*c+a                      The test case regular expression.
xxxxabacccaayyyy               Test case input line 1 (matches).
caxxxxxxxxxxxxxx               Test case input line 2 (matches).
xxxxxxxxxababcaa               Test case input line 3 (matches).
abaaaaaaaaaaaaaa               Test case input line 4.
aaaabaabaccccccc               Test case input line 5.
abaabaccccccccca               Test case input line 6 (matches).

$ java MyApplication < input.txt
xxxxabacccaayyyy
caxxxxxxxxxxxxxx
xxxxxxxxxababcaa
abaabaccccccccca
```

7 Tips and Hints

- A convenient way to read a file line by line in Java is to do it through the standard input using the following lines of code

```
Scanner s = new Scanner(System.in);
while (s.hasNextLine())
    process(s.nextLine());
```

The example run in the previous section describes how to let your program read the content of `input.txt` through standard input.

- Since the objective of the task is to search for the first matching *substring* of a line, you would need to run your automaton from input character 1, then restart the automaton from character 2, and so on. This can be avoided by changing the given regular expression R to the regular expression `.*`R. When the automaton for the latter expression reaches an accepting state, a line is found whose suffix matches R, and thus contains a substring matching R.

- To parse a regular expression in a string `s`, you simply invoke `REParser.parse(s)`. Provided the regular expression in `s` is well formed, the method will return a `RegExp` object representing the root of the AST.

- Note that R`+` and R`?` are equivalent to RR^* and ϵ`|`R respectively.

- Having a graphical representation of your automata may come in handy when debugging your tool. A graphical representation can be generated with the `dot` command (available on the lab computers through `module add csw/1.1`). An example usage of `dot` is given in Figure 1.

```
digraph finite_state_machine {
  rankdir=LR; size="10,10"
  node [shape = doublecircle]; q2;
  node [shape = circle];
  q0 -> q1 [ label = "c" ];
  q0 -> q1 [ label = "a" ];
  q0 -> q2 [ label = "b" ];
  q2 -> q0 [ label = "c" ];
  q2 -> q1 [ label = "c,a" ];
}
```

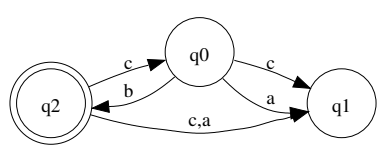

Figure 1: To the left: `graph.gv`, to the right, `graph.png` generated with `dot -o graph.png graph.gv`.